

DeFi Staking Platform

Gas Optimization & Performance Report

This document records the baseline gas measurements for the StakingContract and ERC20Token before optimization, identifies inefficiencies, and outlines the changes applied to reduce on-chain costs.

1. Baseline Gas Usage (Before Optimization)

Measured with Solidity optimizer **disabled** (runs: 200). Tests run: 68 passing across unit and security suites.

StakingContract Methods

Method	Min Gas	Max Gas	Avg Gas	Calls
stake()	87,880	133,154	126,648	57
Withdraw()	67,654	82,054	72,054	12
claimRewards()	63,905	77,585	75,761	15
setRewardRate()	—	—	30,376	4
pauseRewards()	—	—	25,292	3
transferOwnership()	27,210	27,222	27,216	2

ERC20Token Methods

Method	Min Gas	Max Gas	Avg Gas	Calls
approve()	24,758	47,042	46,766	162
transfer()	52,527	52,539	52,534	160
transferOwnership()	27,245	27,269	27,268	69

Deployment Costs

Contract	Deployment Gas	% of Block Limit
ERC20Token	1,272,007	2.1%
StakingContract	1,342,046	2.2%

2. Inefficiencies Identified

1. **stake() avg 126,648 gas — highest cost function.** Storage pointer not cached; stakeAmount and totalStaked written without unchecked blocks despite guaranteed safety.

2. **Struct layout not optimized.** lastUpdateTime stored as uint256 (32 bytes) when uint64 (8 bytes) is sufficient for timestamps, wasting a storage slot.
3. **No unchecked arithmetic.** Solidity 0.8+ adds overflow checks on every arithmetic operation. Several operations are provably safe but still pay the overhead.
4. **_snapshotReward writes to storage even when fresh reward is 0**, paying an unnecessary SSTORE on every transaction.
5. **Optimizer disabled.** hardhat.config.js had optimizer set to false, leaving significant bytecode and runtime savings on the table.
6. **_calculateReward uses checked arithmetic** inside a pure math function where overflow is not realistically possible given token supply constraints.

3. Optimizations Applied

#	Change	Where	Expected Saving
1	uint64 for lastUpdateTime	StakeInfo struct	~200 gas / tx (1 fewer slot)
2	Cache storage pointer (StakeInfo storage index)	stake(), Withdraw()	~100 gas / tx
3	unchecked { += / -= } on safe arithmetic	stake(), Withdraw(), claimRewards()	~50 gas each op
4	Skip storage write if fresh reward == 0	_snapshotReward()	~5,000 gas when no reward
5	unchecked block in _calculateReward	_calculateReward()	~40 gas / call
6	Enable Solidity optimizer (runs: 200)	hardhat.config.js	~10-20% overall reduction

4. Key Code Changes

hardhat.config.js — Enable optimizer:

```
solidity: { version: '0.8.19', settings: { optimizer: { enabled: true, runs: 200 } } }
```

StakingContract.sol — Struct change:

```
Before: uint256 lastUpdateTime // 32 bytes, full slot
After:  uint64 lastUpdateTime // 8 bytes, packs with other fields
```

StakingContract.sol — Unchecked arithmetic in stake():

```
unchecked {
    info.stakeAmount += amount;
    totalStaked += amount;
}
```

StakingContract.sol — Skip write when reward is 0:

```

if (fresh > 0) {
  unchecked { info.pendingRewards += fresh; }
}

```

5. Expected Results After Optimization

Run `$env:REPORT_GAS="true"; npx hardhat test` after applying changes to generate the updated gas table for comparison.

Method	Baseline Avg	Expected Avg	Est. Saving
stake()	126,648	~105,000	~17%
Withdraw()	72,054	~60,000	~17%
claimRewards()	75,761	~63,000	~17%
setRewardRate()	30,376	~25,000	~18%

6. Test Suite Status

Test File	Tests	Status
Staking.test.js	46	All passing
Staking.security.test.js	22	All passing
Total	68	68 passing, 0 failing